Lab Six

ID1063: Introduction to Programming

1. Write a function double calculateArea(double length, double width) that calculates and returns the area of a rectangle.

   In main(): Ask the user to input two floating point numbers, call calculateArea, and print the returned value.

2. Write a function void printBorder(int count) that prints a line of asterisks (*) containing exactly count stars, followed by a newline.

3. Read a string and assume that its length is at least 2. Swap its first two characters and print the modified word.

4. Write a C program to find the index of the first occurrence of a given character in a string. An example is shown below.

   ```
   Input: banana
   character: n
   Output: 2
   ```

   If the character is not present, output -1.

5. Write a C program to check whether a given string is a palindrome (reads the same forwards and backwards). Assume that all characters are English and lowercase.

   ```
   Input:madam
   Output: Palindrome

   Input: hello
   Output: Not a Palindrome
   ```

   Solution idea: Split the steps into logical parts:

   Task 1: Find the length of the string, store it in a variable, say, len

   Task 2: Check Palindrome condition: str[i]==str[len-1-i] ?

   Think about how far i needs to run.

Optional exercises

6. Write a program that accepts an integer n, and n characters from the user, and then stores them in a string. Finally print the string.

   Note: After the n characters, store the null character `'\0'` so that the array forms a valid C string.

7. Read a lowercase word and two lowercase characters, say `x` and `y`. Replace every occurrence of `x` in the word by `y`, and then print the modified word.

   Example

   ```
   Input:
   pepper
   p
   t

   Output:
   tetter
   ```

8. Write a C program to convert all uppercase letters in a string to lowercase and vice versa, **without** using built-in functions like toupper() / tolower(). Example:

   ```
   Input: Hello World
   Output: hELLO wORLD
   ```

9. [**Optional**] Write a C program to count the number of words in a sentence, where words are separated by single spaces. Input: "This is a sample sentence" Output: 5

10. [**Optional**] Write a C program to check whether two given strings are anagrams of each other (contain the same characters with the same frequency, ignoring case and spaces).

    ```
    Input: listen silent
    Output: Yes
    Input:  hello world
    Output: No
    ```

    Hint: Use a frequency array to count character occurrences.

11. [**Optional**] Write a C program to compress a string using counts of repeated consecutive characters. If the compressed string isn't smaller than the original, return the original string. Examples below: note that in the second example, this compression method is not beneficial.

```
Input: aabcccccaaa
Output: a2b1c5a3

Input: abcdef
Output: abcdef
```